

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CANARA ENGINEERING COLLEGE

Programme: **B.E.****CCA: COURSE PROJECT**Semester: **VII**Course Title: **CRYPTOGRAPHY & NETWORK SECURITY**Course Code: **BCS703**Date: **25/08/2026**Submission: **29/10/2026**Section: **A,B,C**

Group	Project No	Mini-Project Title
G1	P1	Linear Congruential Generator (LCG): Implement an LCG with the formula $X_{n+1} = (aX_n + c) \bmod m$ to produce a sequence of pseudo-random numbers.
G2	P2	Elliptic Curve Diffie-Hellman (ECDH): Using the elliptic curve functions, simulate the ECDH key exchange protocol between two parties to establish a shared secret.
G3	P3	Public Key Infrastructure (PKI) Trust Chain Validation: Simulate a three-level certificate chain (Root CA -> Intermediate CA -> End-user). Write a program that verifies the end-user certificate by validating the signature at each step up to the trusted Root CA.
G4	P4	PGP/S/MIME Signature Verification: Write a corresponding program to verify a signed message. It should decrypt the signature with the sender's public key to retrieve the original hash and compare it with a fresh hash of the received message.
G5	P5	Blum Blum Shub (BBS) Generator: Implement the BBS pseudo-random bit generator. The program should take two large prime numbers (p,q) as input and generate a sequence of secure random bits.
G6	P6	Simple Hash Function: Implement one of the "Two Simple Hash Functions" mentioned in the syllabus, for example, a hash based on repeated XOR operations on message blocks.
G7	P7	Hash-based Message Authentication Code (HMAC): Implement HMAC using a standard hash function like SHA-256. The program should be able to generate and verify a MAC for a given message and secret key.
G8	P8	PGP/S/MIME Message Encryption: Create a program that encrypts a message for a recipient. It should: • Generate a random symmetric session key. • Encrypt the message with the session key. • Encrypt the session key with the recipient's public key. • Bundle the encrypted session key and encrypted message together. Also, Write the corresponding program for decryption. The recipient uses their private key to decrypt the session key and then uses the session key to decrypt the main message.
G9	P9	DKIM Signature Verification: Create a program that takes an email with a DKIM signature and a public key (simulating a DNS lookup) to verify the integrity and authenticity of the message.

G10	P10	RSA Key Generation: Write a program that generates a valid RSA key pair. It should: - Find two large prime numbers, p and q. - Calculate $n=pq$ and $\phi(n)=(p-1)(q-1)$. - Choose a public exponent e. - Calculate the private exponent d such that $ed \equiv 1 \pmod{\phi(n)}$.
G11	P11	Symmetric Key Distribution with a KDC: Simulate a Key Distribution Center (KDC). A client (Alice) should be able to request a session key from the KDC to communicate securely with another client (Bob).
G12	P12	Challenge-Response Authentication: Implement a simple remote authentication system where a server sends a random nonce (challenge) to the client, and the client must encrypt it with a shared secret key and send it back (response) to prove its identity.
G13	P13	Transport Layer Security (TLS) Handshake Simulator: Simulate the key steps of a TLS handshake. The program should demonstrate the exchange of ClientHello, ServerHello, server certificate, and the key exchange process (e.g., using RSA or Diffie-Hellman) without implementing the full record protocol.
G14	P14	IPsec ESP Packet Simulation (Transport Mode): Write a program that simulates Encapsulating Security Payload (ESP) in transport mode. It should take a data payload (a mock TCP segment), encrypt it, and prepend an ESP header.
G15	P15	RSA Digital Signature: Implement a basic digital signature scheme using RSA. A function should "sign" a message hash with a private key and another function should "verify" the signature using the corresponding public key.
G16	P16	Symmetric Key Distribution using Asymmetric Crypto: Simulate a scenario where Alice generates a secret session key, encrypts it with Bob's public key, and sends it to Bob so they can communicate symmetrically.
G17	P17	Kerberos AS Exchange Simulation: Simulate the initial exchange with the Kerberos Authentication Server (AS). A client sends its ID to the AS and receives a Ticket-Granting Ticket (TGT) encrypted with its own secret key.
G18	P18	Password-Based Key Derivation (PBKDF2): Implement a simple version of PBKDF2. The program should take a password, a salt, and an iteration count to securely derive a cryptographic key.
G19	P19	IPsec ESP Packet Simulation (Tunnel Mode): Create a simulation of ESP in tunnel mode. The program should take an entire IP packet (header + payload), encrypt it, and then encapsulate it within a new, outer IP packet and an ESP header.
G20	P20	Diffie-Hellman Key Exchange: Simulate a key exchange between two parties, Alice and Bob. They should agree on public parameters (p,g) and independently compute a shared secret key.
G21	P21	X.509 Certificate Parser: Write a program that reads a digital certificate file (e.g., in PEM format) and parses key information like the Subject, Issuer, Validity Period, and Public Key details.
G22	P22	Kerberos TGS Exchange Simulation: Simulate the exchange with the Ticket-Granting Server (TGS). The client presents the TGT to the TGS to request a service ticket for a specific application server

G23	P23	DKIM Email Canonicalization: Implement the "relaxed" canonicalization algorithm for email headers and body as specified by DomainKeys Identified Mail (DKIM).
G24	P24	Security Association (SA) Database: Implement a simple data structure to act as a Security Association Database. It should allow for adding, deleting, and looking up SAs based on a destination address and Security Parameter Index (SPI).
G25	P25	Diffie-Hellman Man-in-the-Middle Attack: Create a simulation with three parties (Alice, Bob, and Mallory). Mallory should intercept the public keys and establish separate shared secrets with Alice and Bob, allowing her to read and modify their messages.
G26	P26	Certificate Signature Verification: Write a function that verifies the authenticity of a certificate. It should take a certificate and the issuer's public key, and check if the signature on the certificate is valid.
G27	P27	PGP/S/MIME Message Signing: Create a program that generates a signed email message. It should compute the hash of the message and encrypt the hash with the sender's private key, then append this signature to the message.
G28	P28	DKIM Signature Generation: Write a program that takes a canonicalized email and a private key to generate a valid DKIM-Signature header for that email.
G29	P29	IPsec Security Policy Engine: Create a simple simulation of an IPsec policy engine. The program should take packet details (source IP, destination IP, protocol) and check them against a predefined policy list to decide whether to permit , deny , or protect (apply IPsec) the packet.
G30	P30	Elliptic Curve Point Addition: Write a program to perform point addition (P+Q) and point doubling (2P) on a given elliptic curve over a finite field F_p . This is the fundamental operation of ECC.
G31	P31	Caesar Cipher with Frequency Analysis: Write a simple program to encrypt and decrypt text using a Caesar cipher (letter shifting). Include a basic frequency analysis feature that counts letter occurrences in an encrypted text to help guess the key automatically.
G32	P32	Simple File Hash Generator : Build a lightweight utility that reads a file and computes its checksum using standard algorithms (like MD5 or SHA-256). The program should compare two files and report whether their content is identical.

Student Groups:

Section A	Section B	Section C
4CB23CS001 & 4CB23CS002 => G1	4CB23CS066 & 4CB23CS067 => G1	4CB23CS127 & 4CB23CS128 => G1
4CB23CS003 & 4CB23CS004 => G2	4CB23CS068 & 4CB23CS069 => G2	4CB23CS129 & 4CB23CS130 => G2
4CB23CS005 & 4CB23CS006 => G3	4CB23CS070 & 4CB23CS071 => G3	4CB23CS131 & 4CB23CS132 => G3
4CB23CS007 & 4CB23CS008 => G4	4CB23CS072 & 4CB23CS073 => G4	4CB23CS133 & 4CB23CS134 => G4
4CB23CS009 & 4CB23CS010 => G5	4CB23CS074 & 4CB23CS075 => G5	4CB23CS135 & 4CB23CS136 => G5
4CB23CS011 & 4CB23CS012 => G6	4CB23CS076 & 4CB23CS077 => G6	4CB23CS137 & 4CB23CS138 => G6
4CB23CS013 & 4CB23CS014 => G7	4CB23CS078 & 4CB23CS079 => G7	4CB23CS139 & 4CB23CS141 => G7
4CB23CS015 & 4CB23CS016 => G8	4CB23CS080 & 4CB23CS081 => G8	4CB23CS142 & 4CB23CS143 => G8
4CB23CS017 & 4CB23CS018 => G9	4CB23CS082 & 4CB23CS083 => G9	4CB23CS144 & 4CB23CS145 => G9
4CB23CS019 & 4CB23CS020 => G10	4CB23CS084 & 4CB23CS085 => G10	4CB23CS146 & 4CB23CS147 => G10

4CB23CS021 & 4CB23CS022 => G11	4CB23CS086 & 4CB23CS087 => G11	4CB23CS148 & 4CB23CS149 => G11
4CB23CS023 & 4CB23CS024 => G12	4CB23CS088 & 4CB23CS089 => G12	4CB23CS150 & 4CB23CS151 => G12
4CB23CS025 & 4CB23CS026 => G13	4CB23CS090 & 4CB23CS091 => G13	4CB23CS152 & 4CB23CS153 => G13
4CB23CS027 & 4CB23CS028 => G14	4CB23CS092 & 4CB23CS093 => G14	4CB23CS154 & 4CB23CS155 => G14
4CB23CS029 & 4CB23CS030 => G15	4CB23CS094 & 4CB23CS095 => G15	4CB23CS156 & 4CB23CS157 => G15
4CB23CS031 & 4CB23CS032 => G16	4CB23CS096 & 4CB23CS097 => G16	4CB23CS158 & 4CB23CS159 => G16
4CB23CS033 & 4CB23CS034 => G17	4CB23CS098 & 4CB23CS099 => G17	4CB23CS160 & 4CB23CS161 => G17
4CB23CS035 & 4CB23CS036 => G18	4CB23CS100 & 4CB23CS101 => G18	4CB23CS162 & 4CB23CS163 => G18
4CB23CS037 & 4CB23CS038 => G19	4CB23CS102 & 4CB23CS103 => G19	4CB23CS164 & 4CB23CS165 => G19
4CB23CS039 & 4CB23CS040 => G20	4CB23CS104 & 4CB23CS105 => G20	4CB23CS166 & 4CB23CS167 => G20
4CB23CS041 & 4CB23CS042 => G21	4CB23CS106 & 4CB23CS107 => G21	4CB23CS168 & 4CB23CS169 => G21
4CB23CS043 & 4CB23CS044 => G22	4CB23CS108 & 4CB23CS109 => G22	4CB23CS170 & 4CB23CS171 => G22
4CB23CS045 & 4CB23CS046 => G23	4CB23CS110 & 4CB23CS111 => G23	4CB23CS172 & 4CB23CS173 => G23
4CB23CS047 & 4CB23CS048 => G24	4CB23CS112 & 4CB23CS113 => G24	4CB23CS174 & 4CB23CS175 => G24
4CB23CS049 & 4CB23CS050 => G25	4CB23CS114 & 4CB23CS115 => G25	4CB23CS176 & 4CB23CS177 => G25
4CB23CS051 & 4CB23CS052 => G26	4CB23CS116 & 4CB23CS117 => G26	4CB23CS178 & 4CB23CS179 => G26
4CB23CS053 & 4CB23CS054 => G27	4CB23CS118 & 4CB23CS119 => G27	4CB23CS180 & 4CB23CS181 => G27
4CB23CS056 & 4CB23CS057 => G28	4CB23CS120 & 4CB23CS121 => G28	4CB23CS182 & 4CB23CS183 => G28
4CB23CS058 & 4CB23CS059 => G29	4CB23CS122 & 4CB23CS123 => G29	4CB23CS184 & 4CB23CS185 => G29
4CB23CS060 & 4CB23CS059 => G30	4CB23CS124 & 4CB23CS125 => G30	4CB23CS186 & 4CB23CS187 => G30
4CB23CS061 & 4CB23CS059 => G31	4CB23CS126 & 4CB24CS400 => G31	4CB23CS188 & 4CB23CS189 => G31
4CB23CS063 & 4CB23CS059 => G32	4CB24CS401 => G32	4CB23CS190 => G32
4CB23CS065 => G33		

Rubrics for Assessment:

Sl. No.	Criteria for Evaluation	Marks
1	Punctuality in completion and submission	5
2	Readability of program/code (Comments, coding standards and best practices)	5
3	Relevance of software requirements, suitability & coverage of Block chain Technology concepts used in the project	5
4	Presentation and Demo	5
5	Brief report and Academic Honesty (Ethics)	5

Instructions to Students:

1. Course project report (max. 8 pages) should be submitted in print format. Prints should be pasted on the assignment book sheets in the sequence (title, description of requirements, Design diagram & code).
2. Do not copy the report contents of other candidates or from online resources. If plagiarism is observed, the report of such candidates will be rejected.
3. The completed report should be submitted to the course instructor on or before **29/10/2026**.
4. Marks distribution is tabulated in the above table, the same criteria will be adapted for evaluation of the project

Signature of Course Instructors with Date	Signature of Course Coordinator with Date
Course module contents are covered completely.	
APPROVED	
Signature of Senior Faculty/Expert with Date	Signature of Senior Faculty/Expert with Date
Signature of Senior Faculty/Expert with Date	Signature of Senior Faculty/Expert with Date
HEAD OF THE DEPARTMENT/CHAIRMAN – MODERATION COMMITTEE	

Rubrics for Assessment:

SI. No.	Criteria for Evaluation	Marks
1	Punctuality in completion and submission	5
2	Readability of program/code (Comments, coding standards and best practices)	5
3	Relevance of software requirements, suitability & coverage of Block chain Technology concepts used in the project	5
4	Presentation and Demo	5
5	Brief report and Academic Honesty (Ethics)	5

Instructions to Students:

- Course project report (max. 8 pages) should be submitted in print format. Prints should be pasted on the assignment book sheets in the sequence (title, description of requirements, Design diagram & code).
- Do not copy the report contents of other candidates or from online resources. If plagiarism is observed, the report of such candidates will be rejected.
- The completed report should be submitted to the course instructor on or before **17/11/2025**.
- Marks distribution is tabulated in the above table, the same criteria will be adapted for evaluation of the project

Signature of Course Instructors with Date	Signature of Course Coordinator with Date
Course module contents are covered completely.	
APPROVED	
Signature of Senior Faculty/Expert with Date	Signature of Senior Faculty/Expert with Date
Signature of Senior Faculty/Expert with Date	Signature of Senior Faculty/Expert with Date
HEAD OF THE DEPARTMENT/CHAIRMAN – MODERATION COMMITTEE	